JETLS.jl 静的解析とランタイム情報を掛け合わせた Julia 言語の高機能 Language Serverの実装

門脇 宗平[1]，**山口 悠地**[2]，森口 草介[2]，渡部 卓雄[2]
(Lead & Core Committer) (Committer)
shuhei.kadowaki@juliahub.com
([1]JuliaHub, Inc. [2]東京科学大学)

JETLS.jl [1]

- **Julia言語** の **高機能 Language Server**
- JuliaHub, Inc. を中心に 2026 年 3 月現在オープンソースで開発中
- 型解析を含むプログラム解析に基づいたエディタ支援を提供
- **マクロ**を含んだり，**具体的な型アノテーションに乏しいジェネリックなプログラム**に対しても解析を行える
- **コンパイラインフラとの資産の共有**によるメンテナンス性の向上

背景

Julia 言語の特徴

- LLVM をバックエンドとした JIT コンパイル
- 多重ディスパッチを中心とした総称関数ベースのスタイル
- 動的な型付け，マクロなどの動的で柔軟な特性
- 豊富なコンパイル時最適化による高速な実行

素早い**プロトタイピング**や**初期の習得コスト低下**を可能にし**科学技術計算**などで広く利用

(静的にさまざまな検査をする言語に比べて)
単純な実装ミスが起きやすい

学習データが非常に豊富といえない言語では
コーディングエージェントへの正確なフィードバックも重要
(Claude Code は LS との統合を提供 [2])

高度なプログラム解析機能を備え早期にフィードバックを行うソフトウェアの強いニーズ

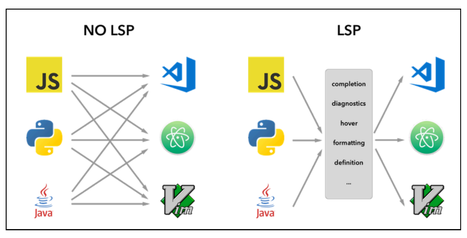
[既存ソフトウェア]

JET.jl [3] ... Julia の型推論プロセスを利用した高度な解析機能を備えたプログラム解析器．
LanguageServer.jl [4] ... 広く普及している Language Server.
基本的な機能はサポートされている一方で高度な解析機能は備えていない

機能と実装

Language Server として実装

Language Server Protocol (LSP) によって通信する開発支援サーバである Language Server として実装し広くさまざまなエディタ上での支援を実現



LSP はエディタとの通信を規定する
([5] より引用)

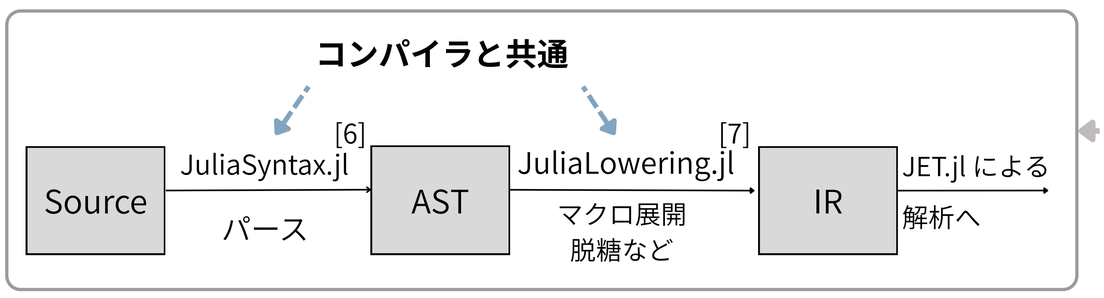
サポートしている主な機能

- 構文エラーの検出
- コード補完
- 定義ジャンプ
- 静的なエラー・性能低下の検出

JET.jl により実現.
既存 LS との機能的に大きな差分

完全な解析の終了時間 ※

package	解析時間 [s]
Example.jl (20 行)	6.06
CSV.jl (3371 行)	29.83
JETLS.jl (13665 行)	71.11



ファイル保存時のエラー検出までに用いられるライブラリ群の一部

※ (Apple M2, 1 thread) で計測．初回の完全な解析にかかる時間

課題点と展望

高精度な解析に実行が必要な場合の扱い (例: マクロ)

課題

Julia コードベースはマクロを多用するスタイルなため
実際のコード実行を伴うマクロ展開をしないと精度よく解析できない ことがよくある
(トップレベル定義も具体的な実行を要求することがありうる)

```
macro config(name)
    val = get(ENV, string(name), nothing)
    val === nothing && error("missing config: $name")
    return :(const $(esc(name)) = $val)
end

@config API_TOKEN
```

このマクロを展開しないと
`API_TOKEN` という変数が存在するかわからない

現状

実際に実行して展開している が **セキュリティリスクを含む**
(信頼できないワークスペースを開いたときに危険)

改善案

ある程度の実行はやはり避けられなさそう． 問題の緩和のために:

案	懸念・欠点
プロセスの権限管理で細かく制御	キャッシュなど処理系自体の機構が動作しなくなる可能性
Wasm ランタイムなどでのサンドボックス実行 [8]	Julia x Wasm 自体がまだ未発展で多くのコードが動作しない
IO，危険な builtin の実行，停止性などを解析して安全なら実行	実用的に問題ないくらい広い範囲のプログラムに対して諸性質を示せるか

呼び出しがない関数の引数型情報の収集と結果の表示

課題

引数の情報がないと有用でない解析結果になりうる

```
function sqsum(xs)
    T = eltype(xs)
    s = zero(T)
    for x in xs
        s += x * x
    end
    return s
end
```

引数に強い型制約をつけないスタイル

実引数の型により実装が大きく異なり，ランタイムまで決まらないこともある．

現状

引数の型アノテーションに基づいて解析

改善案

1. 呼び出し側から引数の制約を求める？
⇨ **Julia の型推論アルゴリズムとは大きく異なる** ので実装コストは高い
2. テストコードなどから収集する？
⇨ **複数の実引数の型が得られたときそれぞれの異なる情報をどう表示すべきか？**
(UI は LSP の担当範囲ではない)

メンテナンス性の向上

課題

1. **長期的なメンテナンス**
2. **最新バージョンへの追従**

は大きなエンジニアリングコストがかかりうる．

現状

コンパイラインフラとの資産の共有 を採用

利点	• 最新バージョンに対応したパーサなどがすぐ使える • 多くの部分では再実装をしないで済む
課題	• コンパイラ内部の安定していない仕様に依存していて，少しの更新で動作しなくなることがある • LS に特化したデータ構造やアルゴリズムの設計がしにくい

改善案

1. **SemanticDB** [9] **のような抽象化レイヤの導入**
2. **コンパイラ仕様の安定化**

などで課題を緩和したい

(これらとコンパイラ開発速度の両立も考慮する必要がある)

そのほかの未実施の改善案

レイテンシ改善

- **ディスク・リモートキャッシュ**を含むキャッシュ機構
- 解析ルーチン自体の改善
  - 軽量な解析への多段階化
  - 解析のストリーム化

プログラマの行動の収集

- LS はかなり広範なユーザが使用することが見込まれる．
- 行動の収集によって言語や LS の改善に役立てたい．(エラー回復や補完のサジェストなど)

(仮に実際に行うことになった際には，当然プライバシーやライセンスなどに十分注意が必要)

[1] https://github.com/aviatesk/JETLS.jl
[2] https://code.claude.com/docs/en/plugins-reference#lsp-servers
[3] https://github.com/aviatesk/JET.jl
[4] https://github.com/julia-vscode/LanguageServer.jl
[5] https://code.visualstudio.com/api/language-extensions/language-server-extension-guide
[6] https://github.com/JuliaLang/JuliaSyntax.jl
[7] https://github.com/JuliaLang/JuliaLowering.jl
[8] https://github.com/rust-lang/compiler-team/issues/876
[9] https://scalameta.org/docs/semanticdb/specification.html